
Ch09

01. 참조

02. 포인터

03. 배열과 포인터

04. 메모리

32bit vs 64bit address체계

- 빌드-구성관리자
- x64 선택
- 64bits 실행파일이 만들어짐
- address체계: 16개의 16진법수로 표기, 0012FEEC3800FEEC

- x86(win32)선택
- 32bits 실행파일이 만들어짐
- address체계: 8개의 16진법수로 표기, 0012FEEC

포인터

- 포인터 자료형은 메모리 위치의 주소를 나타내는 자료형
- 포인터 변수는 포인터 자료를 갖는 변수

변수의 주소

- 프로그램에서는 불, 문자, 정수, 부동 소수점, 클래스 등의 다양한 자료형의 변수를 사용
- 이러한 변수는 1바이트 이상의 메모리를 가짐
- 불(bool)과 문자(char)는 1바이트를 차지하며, 정수(int)는 4bytes, 부동 소수점(double)은 8바이트 를 차지
- sizeof 연산자를 사용하면 몇 바이트를 차지하는지 알 수 있음

주소

- 변수의 주소는 변수가 차지하는 첫 번째 바이트의 주소를 의미
- 주소와 포인터를 기반으로 어떤 작업을 할 때 반드시 기억해야 하는 중요한 내용
- 변수의 주소를 추출하려면 주소 연산자(&)를 사용

예시

- 다음 프로그램을 통해 기본적인 불, 정수, 부동 소수점 자료형의 변수를 하나씩 선언하고 크기, 값, 주소를 확인

주소(5)

프로그램 9-4 변수의 크기, 값, 주소 출력하기

```
1  /*****
2  * The program to define three variables and to print their      *
3  * values and their addresses in the memory                      *
4  *****/
5  #include <iostream>
6  using namespace std;
7
8  int main ( )
9  {
10     // Declaration of three data variables
11     bool flag = true;
12     int score = 92;
13     double average = 82.56;
14     // Printing size, value, and address of the flag variable
15     cout << "A variable of type bool" << endl;
16     cout << "Size: " << sizeof (flag) << " " ;
17     cout << "Value: " << flag << " " ;
18     cout << "Address: " << &flag << endl << endl;
19     // Printing size, value, and address of the score variable
20     cout << "A variable of type int" << endl;
```

주소(6)

프로그램 9-4 변수의 크기, 값, 주소 출력하기

```
21     cout << "Size: " << sizeof (score) << " " ;
22     cout << "Value: " << score << " " ;
23     cout << "Address: " << &score << endl << endl;
24     // Printing size, value, and address of the average variable
25     cout << "A variable of type double" << endl;
26     cout << "Size: " << sizeof (average) << " " ;
27     cout << "Value: " << average << " " ;
28     cout << "Address: " << &average << endl;
29     return 0;
30 }
```

Run:

A variable of type bool

Size: 1 Value: 1 Address: 0x28fef0

A variable of type int

Size: 4 Value: 92 Address: 0x28fef1

A variable of type double

Size: 8 Value: 82.56 Address: 0x28fef5

주소

- `cout << &score` 는 `score`변수의 주소를 출력함
- 주소는 16진법으로 출력
- 주소를 정수 자료형이라고 생각할 수도 있는데, 주소는 이어지는 내용에서 살펴 보겠지만 포인터 자료형이라는 특수한 자료형

주소(8)

연습 조져야할듯

address	variable	Contents				
0012FE60						
0012FE64						
0012FE68						
0012FE6C						int i = 5;
0012FE70						int *ip = &i;
0012FE74	py	0	12	FE	78	double d = 5.7;
0012FE78	y		6			double *dp = &d;
0012FE7C	px	0	12	FE	78	int x = 5;
0012FE80	x		5			int* px = &x;
0012FE84	dp	0	12	FE	88	int y = 6;
0012FE88	rD, d		-6.99			int* py = &y;
0012FE8C						px = py
0012FE90	ip	0	12	FE	94	*px = *py
0012FE94	i		5			double&rD = d;
						rD = -6.99

둘다 int pointer 라서 가능

포인터 자료형과 포인터 변수(1)

- C++은 포인터 자료형과 포인터 변수를 제공
- 이를 활용하면 주소를 조작할 수 있음

포인터 자료형

그냥 주소가 자료형

- 포인터 자료형은 리터럴의 값이 주소인 자료형
- char 포인터(char 변수의 주소), int 포인터(int 변수의 주소), double 포인터(double 변수의 주소) 등의 포인터
- 어떤 자료형의 포인터를 만들 때는 자료형 뒤에 별 기호(*)를 붙임

포인터 자료형과 포인터 변수(2)

- 다음은 몇 가지 자료형에 대한 포인터 자료형을 만든 예시

```
bool*      // A pointer to an object of bool type
int*       // A pointer to an object of int type
double*    // A pointer to an object of double type
Circle*    // A pointer to an object of Circle type
```

포인터 자료형과 포인터 변수(3)

포인터 변수

- 포인터 변수에는 포인터 자료형의 값을 저장할 수 있음
- 어떤 자료형의 포인터라도 포인터 자료형은 4바이트임
- 따라서 포인터 변수는 0x00000000부터 0xFFFFFFFF까지의 주소 저장이 가능

포인터 자료형과 포인터 변수(4)

선언

- 포인터 변수를 선언하려면 어떤 자료형의 포인터를 선언할 것인지 지정해야 함
- 다음은 포인터 선언의 예시
- 포인터 변수의 이름을 pFlag, pScore, pAverage 같은 형태로 붙인 이유는 이 후에 변수들의 주소를 바인드(연결)할 것이기 때문

```
bool*   pFlag;           // pFlag is variable of pointer to bool
int*    pScore;          // pScore is variable of pointer to int
double* pAverage;        // pAverage is variable of pointer to
double
```

포인터 자료형과 포인터 변수(5)

초기화

- 포인터 변수를 선언했다면 다른 변수와 마찬가지로 변수를 사용하기 전에 초기화해야 함
- 이때 리터럴 주소로는 초기화하지 않음
- 반드시 기존에 있는 변수의 주소로 초기화해야 함
- 따라서 주소 연산자(&)와 변수를 조합한 형태를 사용
- 다음은 3개의 변수를 초기화하는 예시

포인터 자료형과 포인터 변수(6)

```
// Initialize pFlag with address of flag
bool* pFlag = &flag;
// Initialize pScore with address of score
int* pScore = &score;
// Initialize pAverage with address of average
double* pAverage = &average;
```

- 예로 pFlag는 불 자료형에 대한 포인터이므로, 불 자료형의 변수를 기반으로 초기화해야 함

포인터 자료형과 포인터 변수(7)



그림 9-9 포인터 변수와 포인트된 변수

예시

- 다음 코드는 포인터 변수에 포인터 리터럴을 직접 할당할 수 없으므로 컴파일 오류를 발생시킴

```
double* pAverage = 0x123467; // Error. No literal addresses
```


포인터 자료형과 포인터 변수(8)

예시

- 다음 코드도 컴파일 오류가 발생하며 double 자료형에 대한 포인터 변수에 int 자료형 변수의 주소를 할당할 수 없음

```
int num;
double* p = &num;    // Error. Variable p is of type double*
```

간접 참조

dereferencing : 주소에 접근해서 값을 변경

* : dereferencing operator (!= pointer *)

- 포인터 변수를 선언하고 초기화했다면 포인터 변수에 저장된 주소를 활용해서 포인터된 변수의 값에 접근할 수 있음
- 이때는 * 기호를 사용하는 간접 참조 연산자(indirection operation)를 사용.
dereferencing operator

```
*pFlag;    // value stored in flag
*pScore;   // value stored in score
*pAverage; // value stored in average
```

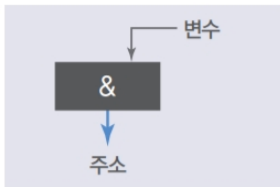
포인터 자료형과 포인터 변수(9)

연산자 정리

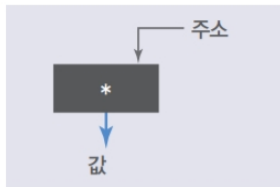
- 지금까지 설명했던 연산자가 2개 있으며 바로 주소 연산자와 간접 참조 연산자
address of operator, dereferencing operator

표 9-2 주소 표현식

그룹	이름	연산자	표현식	우선 순위	결합방향
단항	주소	&	&lvalue	17	←
	간접 참조	*	*pointer		



(a) 주소 연산자



(b) 간접 참조 연산자

그림 9-10 주소 연산자와 간접 참조 연산자

포인터 자료형과 포인터 변수(10)

- C++은 & 기호와 * 기호를 [표 9-3]처럼 3가지 상황에 사용

표 9-3 & 기호와 * 기호의 사용

기호	자료형 정의	단항 연산자	이항 연산자
&	자료형&	&변수	x & y
*	자료형*	*변수	x * y

포인터 자료형과 포인터 변수(11)

- & 기호는 참조 자료형을 정의할 때 사용. 변수의 주소를 얻기 위한 단항 연산자로 사용할 수도 있음. 마지막으로 비트 AND 연산자라는 이항 연산자로 사용될 수 있음
- 참조 자료형을 정의할 때는 자료형 뒤에 int& 형태로 배치. 변수의 주소를 얻을 때는 변수 이름 앞에 붙임(&score). 이항 연산자로 사용할 때는 두 피연산자 사이에 사용
- 포인터 자료형을 정의할 때는 자료형 뒤에 int* 형태로 배치
- 간접 참조할 때는 포인터 앞에 붙임(*pScore)
- 이항 연산자로 사용할 때는 두 피연산자 사이에 사용

값 추출(1)

- 포인터 관계가 만들어지면, 데이터 변수 또는 포인터 변수를 사용해서 메모리 위치에 저장된 값에 접근할 수 있음
- [프로그램 9-5]는 직접 참조와 간접 참조를 사용해서 값에 접근하는 방법을 보여주는 예시

프로그램 9-5 직접 참조와 간접 참조

```
1  /*****
2  * The program shows how to access (retrieve or change) the value *
3  * of a data variable both directly and indirectly.                *
4  *****/
5  #include <iostream>
6  using namespace std;
7
8  int main ( )
9  {
10     // Declaration and initialization of variables
11     int score = 92;
12     int* pScore = &score;
13     // Retrieving value of data variable directly and indirectly
14     cout << "Direct retrieve of score: "<< score << endl;
15     cout << "Indirect retrieve of score: "<< *pScore;
16     return 0;
17 }
```

Run:

```
Direct retrieve of score: 92
Indirect retrieve of score 92
```

특수한 포인터(1)

- C++에는 특수한 포인터로 아무 것도 가리키지 않는 포인터와 모든 것을 가리킬 수 있는 포인터가 있음
- 이번 절에서는 이와 관련된 내용을 간단하게 살펴봄

널 포인터

- 아무 것도 가리키지 않는 포인트를 널 포인터라고 함
- C 언어는 NULL이라는 단어를 사용해서 널 포인터를 정의하는 것이 일반적이지만, C++에서는 리터럴 0을 사용하는 것이 일반적
- 따라서 널 포인터는 다음과 같은 방법으로 생성

```
int* p1 = NULL;  
double* p2 = NULL;
```

특수한 포인터

- 포인터는 널 포인터일 때 false로 변환되고, 이외의 경우에는 true로 변환
- 따라서 다음과 같이 조건문을 사용해서 포인터가 널 포인터인지 아닌지 확인할 수 있음

```
int x = 7;  
int* p = &x;  
if (p) {...}    // Test is true here (p is not null)  
p = NULL;  
if (p) {...}    // Test is false here (p is null)
```


특수한 포인터(4)

범용 포인터

- void 포인터는 범용 포인터(generic pointer)로서 모든 자료형의 객체를 가리킬 수 있음
- 다음은 범용 포인터를 사용해서 여러 자료형의 객체를 가리키는 예시

```
void* p;           // Declaring a void pointer
int x = 10;
p = &x;            // Make p to point to an int type
double y = 23.4;
p = &y;            // Make p to point to a double type
```

포인터의 활용(1)

- 참조처럼 포인터도 함수 사이의 통신에 활용할 수 있음
- 이때 활용은 크게 매개변수로 포인터를 전달하는 포인터로 전달(pass-by-pointer)과 포인터를 리턴하는 포인터로 리턴(return-by-pointer)이 있음

포인터로 전달

- 함수가 함수를 호출할 때 포인터를 사용해서 데이터를 전달할 수 있음
- 다음 그림은 값으로 전달, 참조로 전달, 포인터로 전달이라는 메커니즘을 정리한 것

포인터의 활용(2)



그림 9-14 함수 데이터 전달의 3가지 메커니즘 비교

포인터의 활용(3)

2가지를 비교할 수 있게 간단한 프로그램을 작성

doIt 함수는 정수를 받아서 어떤 처리를 하는 함수

왼쪽은 값으로 전달, 오른쪽은 포인터로 전달

```
#include <iostream>
using namespace std;
// Prototype
void doIt(int) ;
int main()
{
    int num1 = 10;
    doIt(num1) ;
    return 0;
}
// Pass-by-value
void doIt(int num2)
{
    // Code
}
```

```
#include <iostream>
using namespace std;
// Prototype
void doIt(int*) ;
int main()
{
    int num = 10;
    doIt(&num) ;
    return 0;
}
// Pass-by-pointer
void doIt(int* pNum)
{
    // Code
}
```

포인터의 활용(4)

특징

- 포인터로 전달에서 매개변수(parameter)는 인수(argument)의 주소
- 매개변수와 인수는 내부적으로 다음과 같이 전달

```
int* pNum = &num;
```

포인터의 활용(5)

- 포인터로 전달은 인수와 매개변수가 연결
- 포인터로 전달은 4바이트의 포인터만 복사하면 되므로 복사 비용이 굉장히 적음
- 클래스 자료형의 객체처럼 큰 객체를 함수로 전달할 때 사용하면 좋음

포인터의 활용(6)

사용 방법

- 지금까지 살펴보았던 값으로 전달, 참조로 전달, 포인터로 전달이라는 3가지 메커니즘을 어떻게 사용하는지 정리해보겠습니다
- 변경을 하면 안 되는 경우에 작은 객체라면 값으로 전달을 사용
- 큰 객체라면 `const` 한정자를 붙여서 참조로 전달 또는 포인터로 전달을 사용
- 변경이 필요한 경우에는 반드시 참조로 전달 또는 포인터로 전달을 사용

포인터의 활용(7)

예시

- 두 데이터를 스왑하는 함수를 만든다면 변경이 필요
- 반드시 참조로 전달 또는 포인터로 전달을 사용
- 이전에 참조로 전달로 어떻게 스왑할 수 있는지 살펴보았으므로, 이번에는 포인터로 전달을 사용해서 어떻게 스왑하는지 살펴봄
- 코드는 [프로그램 9-6]과 같음

포인터의 활용(8)

프로그램 9-6 포인터로 전달을 사용해서 스왑하는 함수

```
1  /*****
2  * The program shows how to use pass-by-pointer to allow a      *
3  * called function to swap two values in the calling function. *
4  *****/
5  #include <iostream>
6  using namespace std;
7
8  void swap (int* first, int* second) ; // Prototype
9
10 int main ( )
11 {
12     // Definition of two variables
13     int x = 10;
14     int y = 20;
15     // Printing the value of x and y before swapping
16     cout << "Values of x and y before swapping." << endl;
17     cout << "x: " << x << " " << "y: " << y << endl;
18     // Calling swap function to swap the values of x and y
19     swap (&x , &y);
20     // Printing the value of x and y after swapping
```

포인터의 활용(9)

프로그램 9-6 포인터로 전달을 사용해서 스왑하는 함수

```
21     cout << "Values of x and y after swapping." << endl;
22     cout << "x: " << x << " " << "y: " << y;
23     return 0;
24 }
25 /*****
26  * The swap function swaps the values of parameters and      *
27  * pass-by-pointer allows the corresponding arguments in main *
28  * to be swapped accordingly.                                  *
29  *****/
30 void swap (int* pX, int* pY)
31 {
32     int temp = *pX;
33     *pX= *pY;
34     *pY = temp;
35 }
```

Run:

Values of x and y before swapping.

x: 10 y: 20

Values of x and y after swapping.

x: 20 y: 10

포인터의 활용(10)

포인터로 리턴

- 호출된 함수에서 호출한 함수 쪽으로 포인터를 리턴할 때 사용

특징

- 포인터로 리턴은 다음 프로토타입처럼 매개변수로 포인터를 받아서 포인터를 리턴하는 형태로 구성

```
type* function(type* ...);
```

포인터의 활용(11)

예시

- 두 정수 중에 더 큰 정수를 찾는 함수를 만든다고 가정
- 값으로 전달과 값으로 리턴 조합, 포인터로 전달과 포인터로 리턴 조합을 사용해서 각각 프로그램을 작성하면 다음과 같음

포인터의 활용(12)

```
#include <iostream>
using namespace std;

int larger(int x, int y)
{
    if (x > y)
    {
        return x;
    }
    return y;
}

int main()
{
    int x = 10;
    int y = 20;
    int z = larger(x, y);
    cout << z;
    return 0;
}
```

```
#include <iostream>
using namespace std;

int* larger(int* x, int* y)
{
    if (*x > *y)
    {
        return x;
    }
    return y;
}

int main()
{
    int x = 10;
    int y = 20;
    //int z = *larger(&x, &y);
    //cout << z;
    int* p = larger(&x, &y);
    cout << *p << endl;
    return 0;
}
```